

1. Core Problem During Migration

When you:

- Restore DB OR
- Attach DB to another server

Users inside DB come, but server logins do NOT

Result:

Users cannot login

Error: *"User is not associated with a trusted SQL Server connection"*

Orphaned users

2. Key Concepts

Level	Object
Server Level	Login
Database Level	User

They are linked by **SID (Security Identifier)**

3. Practical Scenario

Situation:

You migrated database SchoolDB to a new server.

Now users cannot access it.

4. Step-by-Step Solution

Step 1: Find Orphaned Users

```
USE SchoolDB;
```

```
GO
```

```
EXEC sp_change_users_login 'Report';
```

Shows users without matching login

Step 2: Fix Orphaned Users (Method 1 – Quick Fix)

```
EXEC sp_change_users_login 'Auto_Fix', 'username';
```

Automatically creates login and maps user

Note:

- Works only for **SQL logins**

5. Modern Method (Recommended)

Step 1: Create Login with SAME SID

On **old server**, run:

```
SELECT name, sid
```

```
FROM sys.sql_logins;
```

Step 2: Recreate Login on new server

```
CREATE LOGIN username
```

```
WITH PASSWORD = 'StrongPassword123',
```

```
SID = 0xABCDEF123456; -- same SID
```

Step 3: Map User

```
USE SchoolDB;
```

```
GO
```

ALTER USER username WITH LOGIN = username;

6. Best Method (Real World) – Script Logins

Use Microsoft stored procedure:

EXEC sp_help_revlogin;

This generates:

- Login
- Password hash
- SID

Run script on new server → exact same logins created

7. Practice Tasks

Q1:

Find orphaned users in database CompanyDB

Q2:

Fix user emp_user using modern method

Q3:

Write query to map user sales_user to login sales_login

Q4:

What happens if SID is different between login and user?

8. Migration Scenarios

Scenario A: Restore DB to new server

You MUST migrate logins separately

Scenario B: Detach/Attach

Same issue → logins missing

Scenario C: Full server migration

Use:

- sp_help_revlogin
 - Backup master DB (advanced)
-

9. Role & Permission Migration

Export permissions

SELECT

```
dp.name AS UserName,
dp.type_desc,
o.name AS ObjectName,
p.permission_name,
p.state_desc
```

FROM sys.database_permissions p

JOIN sys.objects o ON p.major_id = o.object_id

JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id;

Example: Grant permission

```
GRANT SELECT ON dbo.Students TO username;
```

10. Hands-On Lab

Step 1: Create login & user

```
CREATE LOGIN test_user WITH PASSWORD = 'Test@123';  
GO
```

```
USE SchoolDB;
```

```
CREATE USER test_user FOR LOGIN test_user;
```

Step 2: Backup & Restore DB to another server

Step 3: Check orphan

```
EXEC sp_change_users_login 'Report';
```

Step 4: Fix

```
ALTER USER test_user WITH LOGIN = test_user;
```

The login was created successfully, but in Microsoft SQL Server a login alone does **not automatically get access to any database**.

You need to:

1. Create a user inside the database
2. Map the login to that database
3. Grant permissions

Example:

```
-- Select the database
```

```
USE YourDatabaseName;
```

```
GO
```

```
-- Create database user mapped to login
```

```
CREATE USER test_user FOR LOGIN test_user;
```

```
GO
```

```
-- Give access/roles
```

```
ALTER ROLE db_owner ADD MEMBER test_user;
```

```
GO
```

You can replace db_owner with:

- db_datareader → read only
 - db_datawriter → insert/update/delete
 - db_ddladmin → create/alter tables
-

If the error says:

Cannot open database requested by the login.

Login failed for user 'test_user'

then either:

- the default database does not exist, or
- the user is not mapped to the database.

Fix it using:

```
ALTER LOGIN test_user
```

```
WITH DEFAULT_DATABASE = master;  
GO
```

or set another valid database.

You can also grant access using SSMS GUI:

1. Open SSMS
2. Security → Logins
3. Right click test_user
4. Properties
5. User Mapping
6. Tick the database
7. Select roles (db_owner etc.)
8. OK

After that, reconnect using test_user.

Final Takeaway

- **Database migration ≠ Security migration**
 - Always migrate:
 - ✓ Logins
 - ✓ Users
 - ✓ Roles
 - ✓ Permissions
-